



Saad Mughal | F2023-009
Sheikh Muhammad Ibrahim | F2023-630

Khadijah Zahoor | F2023-956

Total Marks: 10

Due 4 January 2026

Operating Systems

RetroPie

Project Documentation

Course Lecturer: Noreen Khalid

Lab Instructor: Ali Hafeez Malik

Section: C

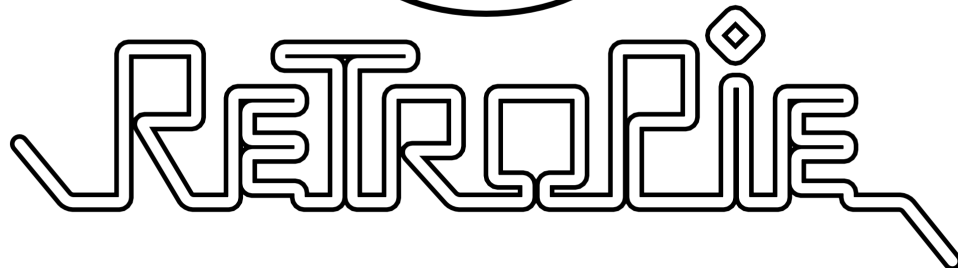


Table of Contents

1. Project Overview.....	3
1.1 Project Title.....	3
1.2 Team Members.....	3
1.3 Project Abstract.....	3
1.4 Project Objectives.....	3
2. Operating System Architecture.....	4
2.1 System Architecture.....	4
2.2 Key Operating System Concepts Demonstrated.....	4
Process Management.....	4
Memory Management.....	4
File System Management.....	5
Device Drivers & Hardware Abstraction.....	5
I/O Management.....	5
Boot Process.....	5
System Services & Daemons.....	6
3. System Requirements.....	6
3.1 Hardware Specifications.....	6
3.2 Software Components.....	7
4. Implementation Process.....	7
4.1 Installation Workflow.....	7
Step 1: Preparing the Operating System Image.....	7
Step 2: Flashing microSD Card.....	8
Step 3: Initial Boot and Configuration.....	11
Step 4: Controller Configuration.....	11
4.2 ROM Transfer and Organization.....	12
Network Transfer via SSH/SCP.....	12
File System Organization.....	13
4.3 System Configuration.....	14
Performance Optimization.....	14
5. Implementation Challenges.....	14
6. Testing and Performance Metrics.....	15
6.1 System Performance.....	15
6.2 Functional Testing.....	15
7. Learning Outcomes.....	16
7.1 Technical Knowledge Gained.....	16
7.2 Operating System Concepts Applied.....	16
8. Conclusion.....	17
8.1 Future Enhancements.....	17

9. References.....	17
10. Appendix.....	17

RetroPie Gaming Console on Raspberry Pi

Operating Systems Course Project Documentation

1. Project Overview

1.1 Project Title

RetroPie Gaming Console: A Custom Linux-Based Retro Gaming Operating System

1.2 Team Members

- Saad Mughal - F2023-009
- Sheikh Muhammad Ibrahim - F2023-630
- Khadijah Zahoor - F2023-956

1.3 Project Abstract

RetroPie is a Linux-based retro gaming operating system built on Raspberry Pi OS that transforms a Raspberry Pi into a dedicated gaming console. This project demonstrates core operating system concepts including process management, file system organization, device driver configuration, and system service administration. Implementation involves installing RetroPie on Raspberry Pi 4 hardware, configuring emulation software (RetroArch, EmulationStation), optimizing system performance, and managing ROM libraries. The project showcases practical applications of OS architecture, boot processes, I/O management, and hardware abstraction layers in a real-world embedded Linux environment.

1.4 Project Objectives

- Install and configure RetroPie OS on Raspberry Pi 4 hardware
 - Demonstrate Linux-based operating system architecture and components
 - Implement system-level configurations for gaming performance optimization
 - Configure device drivers, I/O systems, and hardware abstraction
 - Document hardware-software interaction and OS service management
-

2. Operating System Architecture

2.1 System Architecture

RetroPie runs on Raspberry Pi OS (Debian-based Linux) with the following layered architecture:

User Interface Layer
EmulationStation Frontend
Application Layer
RetroArch & Emulator Cores (Game Console Emulation)
System Services Layer
systemd, Samba, SSH, Network Services
Operating System Layer
Raspberry Pi OS (Debian Linux)
Kernel Layer
Linux Kernel (ARM Architecture)
Hardware Layer
Raspberry Pi Hardware (CPU, GPU, RAM, Storage, I/O)

2.2 Key Operating System Concepts Demonstrated

Process Management

- Emulators run as independent processes with dedicated resource allocation
- Multi-threading utilized for frame rendering and audio processing
- CPU scheduling optimized for real-time gaming performance (60 FPS target)
- Process priority management for system vs. emulation processes

Memory Management

- RAM allocation configured per emulator based on console requirements
- GPU memory split optimized (256MB GPU / 3.7GB system on 4GB Pi 4)
- Virtual memory management for large ROM loading

File System Management

- ext4 file system on microSD card for OS and data
- ROM organization: /home/pi/RetroPie/roms/[system]/
- Configuration files: /opt/retropie/configs/
- BIOS files: /home/pi/RetroPie/BIOS/

Device Drivers & Hardware Abstraction

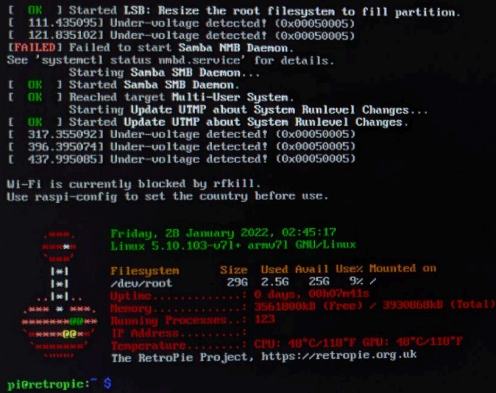
- VideoCore VI GPU drivers for hardware-accelerated graphics
- USB/Bluetooth controller input device drivers
- ALSA audio subsystem for sound output
- HDMI display interface drivers

I/O Management

- Real-time controller input processing with minimal latency
- Video output buffering for 60Hz refresh rate synchronization
- Storage I/O optimization for ROM loading and save states

Boot Process

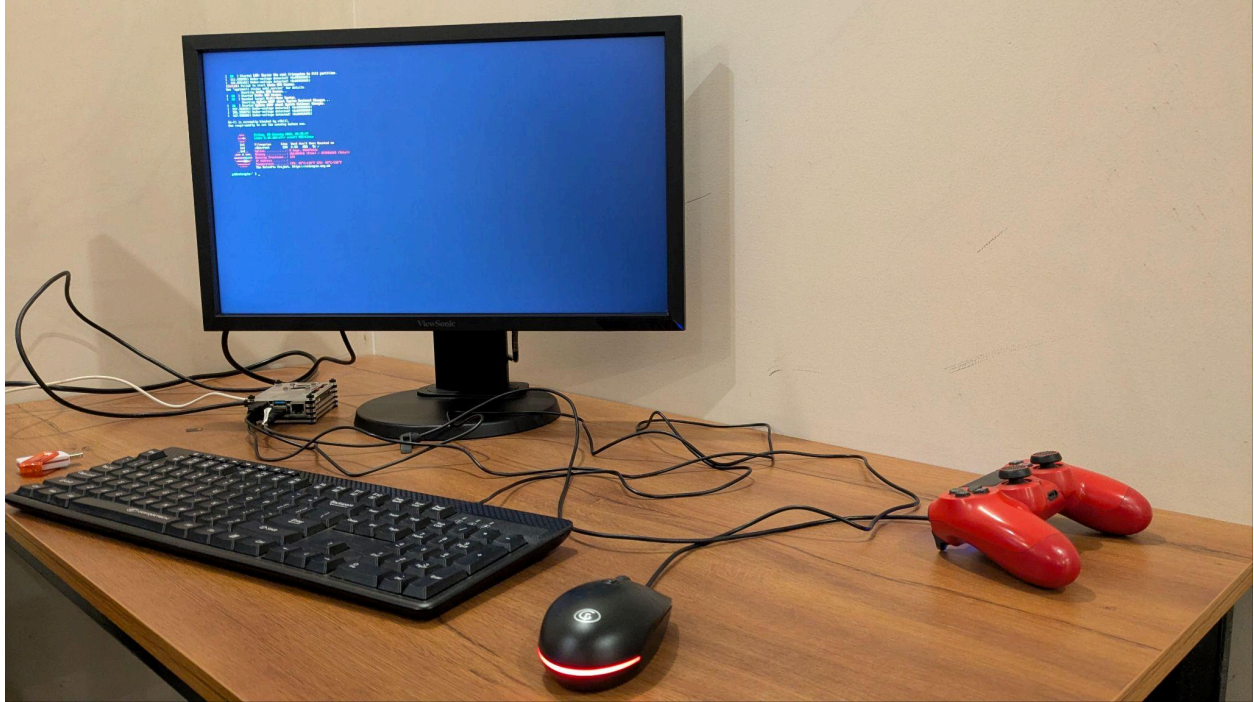
- Raspberry Pi firmware bootloader initialization
- Linux kernel loading and hardware detection
- systemd init system startup and service orchestration
- Auto-start configuration for EmulationStation on boot



System Services & Daemons

- ### 3. System Requirements

Component	Specification
Main Board	Raspberry Pi 4 Model B, 4GB RAM
Storage	32GB microSD Card (Class 10, UHS-I)
Power Supply	5V/3A USB-C Power Adapter
Display	HDMI Monitor/TV (1080p)
Input Devices	USB Keyboard, USB Mouse, USB/Bluetooth Game Controller
Connectivity	WiFi 802.11ac/ Gigabit Ethernet



[Fig 3: Physical Setup]

3.2 Software Components

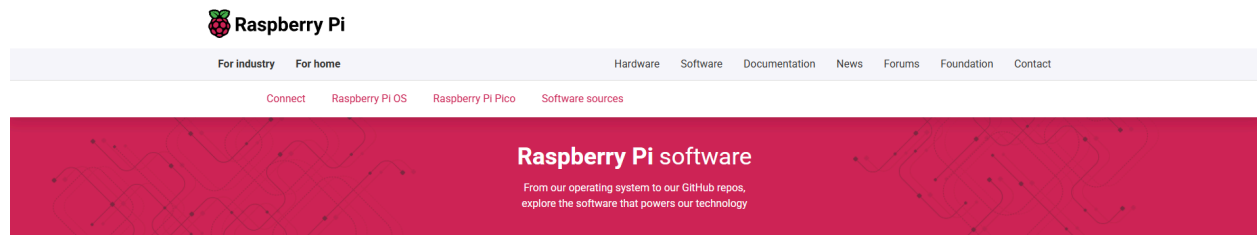
- RetroPie Version: 4.8 (released March 2022)
- Base OS: Raspberry Pi OS (Debian 11 Bullseye)
- Linux Kernel: 5.10.103-v7l+ (ARM architecture)
- Frontend: EmulationStation
- Emulation Framework: RetroArch with libretro cores

4. Implementation Process

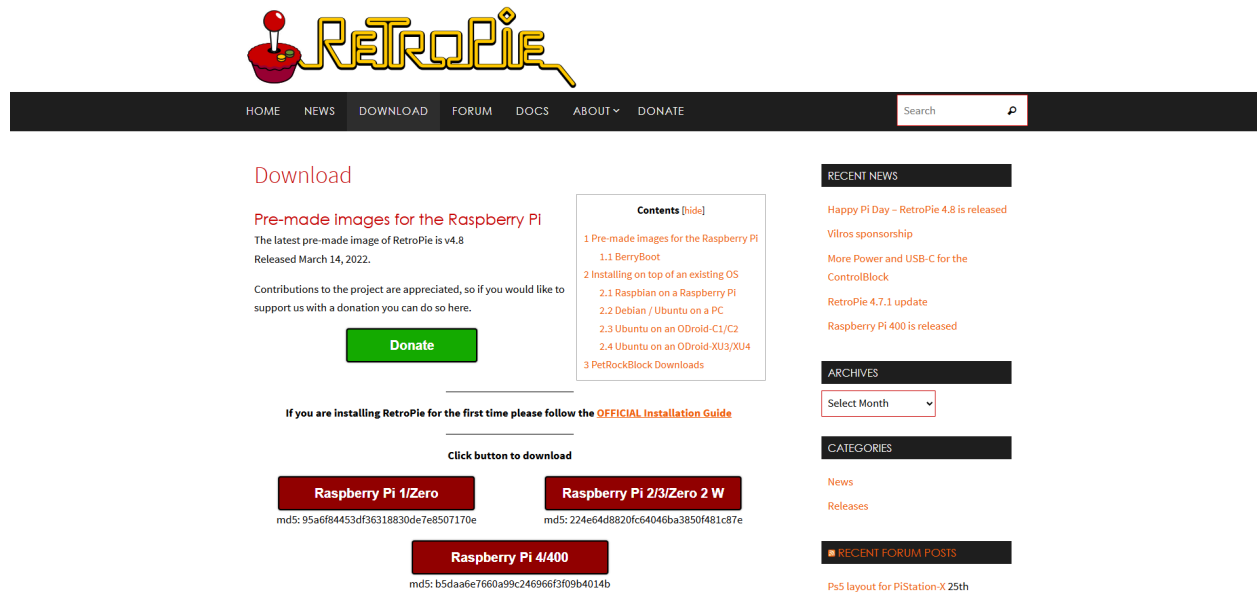
4.1 Installation Workflow

Step 1: Preparing the Operating System Image

- Downloaded RetroPie 4.8 image from official repository (retropie.org.uk)
- Downloaded Raspberry Pi Imager tool for Windows
- Verified image integrity using MD5 checksum



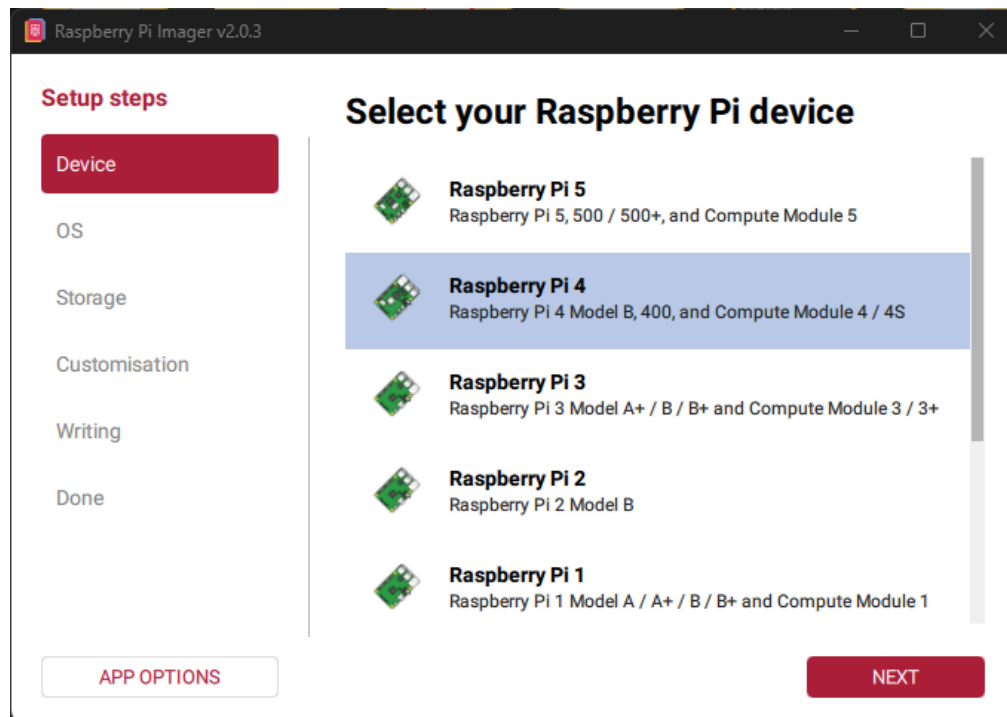
[Fig 4: Raspberry Pi Imager Download Page]



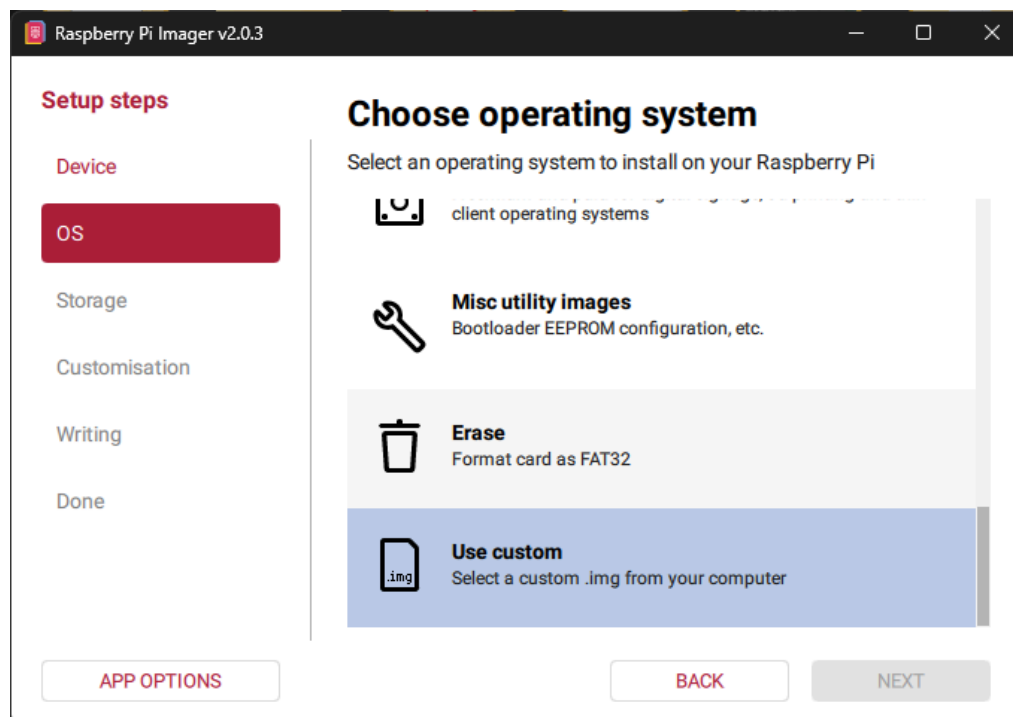
[Fig 5: RetroPie Download Page]

Step 2: Flashing microSD Card

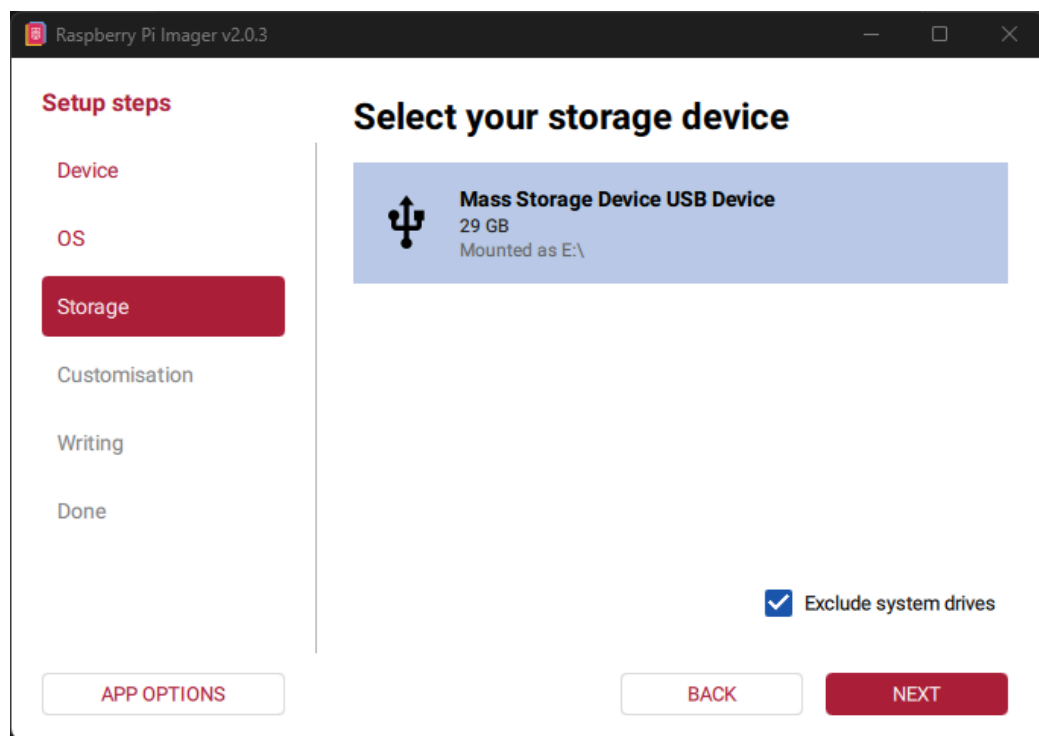
- Launched Raspberry Pi Imager application
- Selected Raspberry Pi 4 as target device
- Selected 'Use custom' option and loaded RetroPie .img file
- Selected 32GB microSD card as storage device
- Initiated write process (completion time: ~8 minutes)



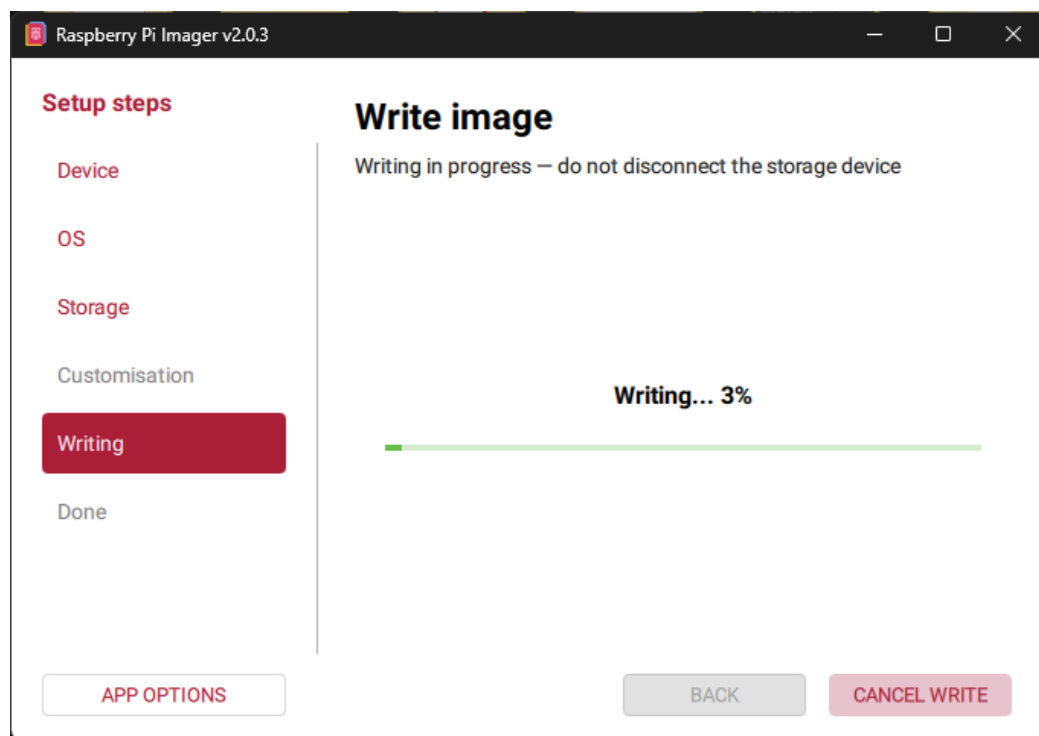
[Fig 6: Raspberry Pi Imager - Device Selection]



[Fig 7: Raspberry Pi Imager - OS Selection]



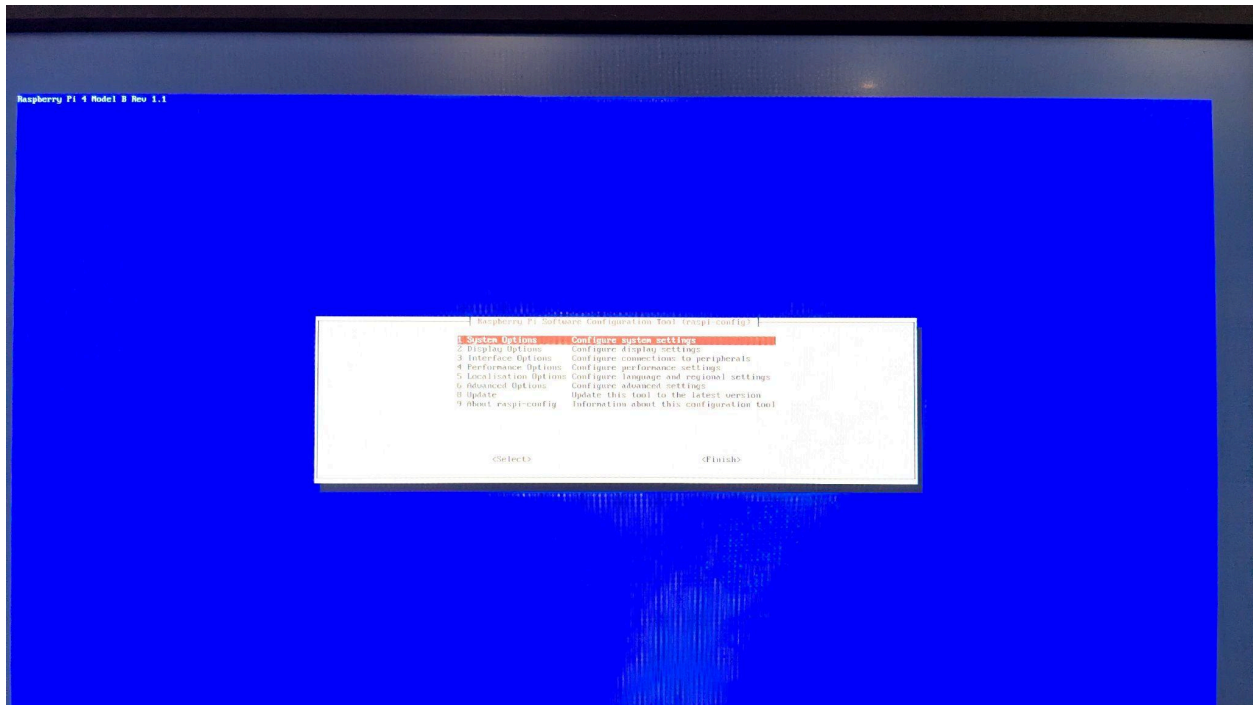
[Fig 8: Raspberry Pi Imager - Storage Selection]



[Fig 9: Raspberry Pi Imager - Writing Progress]

Step 3: Initial Boot and Configuration

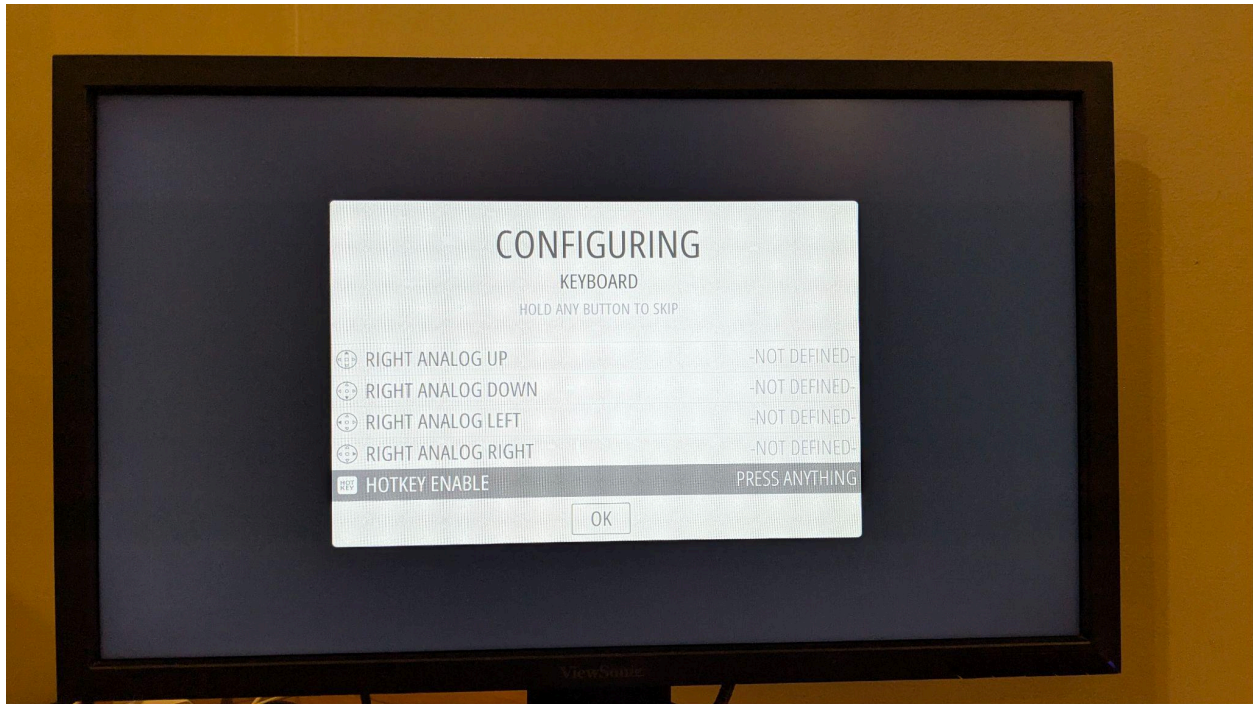
- Inserted microSD card into Raspberry Pi 4
- Connected HDMI display, keyboard, mouse, and power
- First boot initiated automatic file system expansion
- Configured WiFi network via raspi-config utility
- Enabled SSH service for remote administration



[Fig 10: raspi-config Menu]

Step 4: Controller Configuration

- EmulationStation detected USB controller on first launch
- Configured button mappings: D-pad, A/B/X/Y, L/R triggers, Start/Select
- Keyboard configured as secondary input device



[Fig 11: EmulationStation Controller Configuration]

4.2 ROM Transfer and Organization

Network Transfer via SSH/SCP

- Identified Raspberry Pi IP address: 192.168.1.14
- Verified network connectivity using ping command
- Transferred ROM files using SCP protocol from Windows host
- Command: `scp -r D:\Games\roms\* pi@192.168.1.14:~/RetroPie/roms/`

```
PowerShell
PowerShell 7.5.4
PS C:\Users\saadm> ping 192.168.1.14

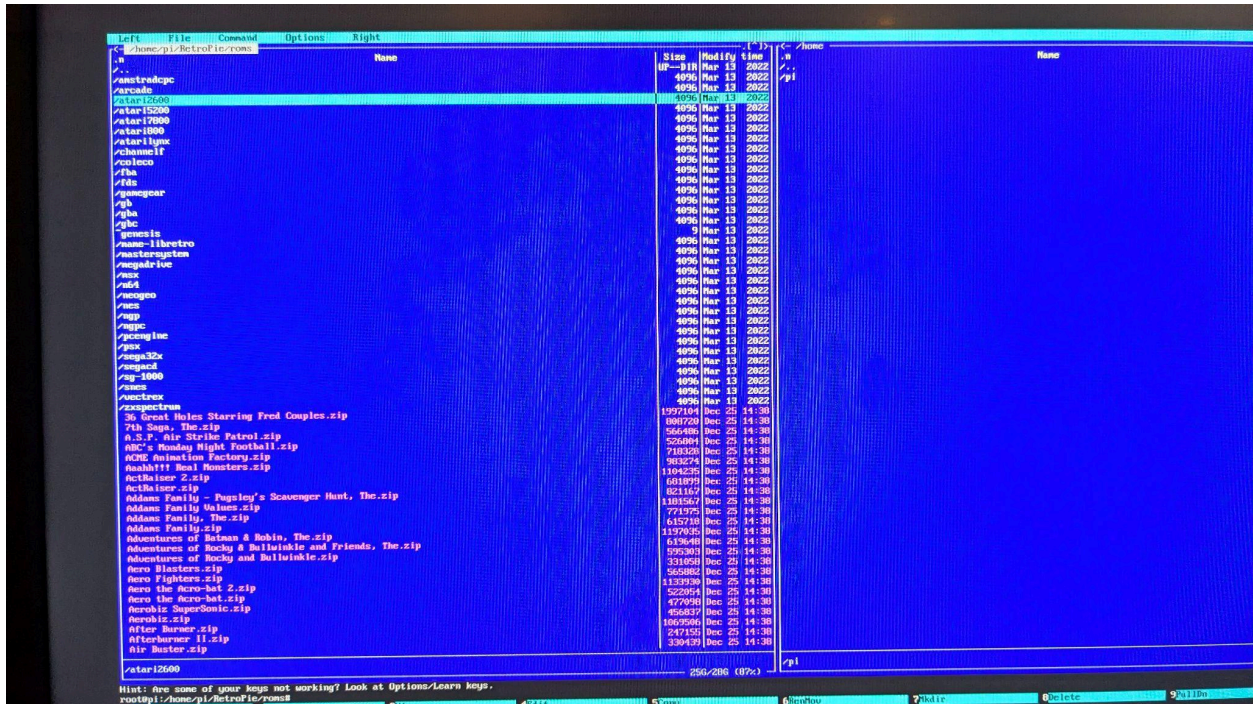
Pinging 192.168.1.14 with 32 bytes of data:
Reply from 192.168.1.14: bytes=32 time=21ms TTL=64
Reply from 192.168.1.14: bytes=32 time=2ms TTL=64
Reply from 192.168.1.14: bytes=32 time=2ms TTL=64
Reply from 192.168.1.14: bytes=32 time=2ms TTL=64

Ping statistics for 192.168.1.14:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 21ms, Average = 6ms
PS C:\Users\saadm> scp -r D:\Games\roms\* pi@192.168.1.14:~/RetroPie/roms/
pi@192.168.1.14's password:
36 Great Holes Starring Fred Couples.zip          100% 1950kB  6.9MB/s  00:00
7th Saga, The.zip                                100% 790kB   7.1MB/s  00:00
A.S.P. Air Strike Patrol.zip                      100% 553kB   7.3MB/s  00:00
ABC's Monday Night Football.zip                  100% 510kB   4.6MB/s  00:00
ACME Animation Factory.zip                       100% 701kB   4.5MB/s  00:00
Aaahh!!! Real Monsters.zip                       100% 960kB   5.0MB/s  00:00
ActRaiser 2.zip                                  100% 1078kB  2.6MB/s  00:00
ActRaiser.zip                                    100% 666kB   1.8MB/s  00:00
Addams Family - Pugsley's Scavenger Hunt, The.zip 100% 802kB   5.8MB/s  00:00
Addams Family Values.zip                         100% 1154kB  7.4MB/s  00:00
Addams Family, The.zip                          100% 754kB   6.5MB/s  00:00
Addams Family.zip                               100% 681kB   4.1MB/s  00:00
Adventures of Batman & Robin, The.zip            100% 1169kB  5.2MB/s  00:00
Adventures of Rocky & Bullwinkle and Friends, The.zip 100% 695kB   4.4MB/s  00:00
Adventures of Rocky and Bullwinkle.zip          100% 581kB   4.6MB/s  00:00
Aero Blasters.zip                               100% 323kB   4.0MB/s  00:00
Aero Fighters.zip                               100% 553kB   4.0MB/s  00:00
Aero the Acro-bat 2.zip                         100% 1197kB  5.4MB/s  00:00
Aero the Acro-bat.zip                           100% 510kB   4.7MB/s  00:00
Aerobiz SuperSonic.zip                          100% 466kB   5.6MB/s  00:00
Aerobiz.zip                                    100% 446kB   2.9MB/s  00:00
After Burner.zip                               100% 1044kB  5.7MB/s  00:00
Afterburner II.zip                             100% 241kB   3.8MB/s  00:00
Air Buster.zip                                 100% 323kB   4.6MB/s  00:00
Air Busters.zip                                100% 314kB   4.4MB/s  00:00
Air Cavalry.zip                                100% 459kB   5.9MB/s  00:00
Air Driver.zip                                 100% 282kB   4.9MB/s  00:00
Aladdin.zip                                    100% 1160kB  7.4MB/s  00:00
Alfred Chicken.zip                             100% 721kB   6.2MB/s  00:00
Alien 3.zip                                    100% 313kB   3.6MB/s  00:00
Alien Soldier.zip                              100% 1113kB  6.9MB/s  00:00
Alien vs. Predator.zip                         100% 581kB   5.8MB/s  00:00
Alien-3.zip                                    100% 589kB   7.3MB/s  00:00
Alisia Dragoon.zip                             100% 597kB   6.0MB/s  00:00
Ambition of Caesar.zip                         100% 387kB   3.2MB/s  00:00
American Gladiators.zip                        100% 706kB   4.6MB/s  00:00
```

[Fig 12: PowerShell - Ping and SCP Transfer]

File System Organization

- SSH connection established to Raspberry Pi
- Created system-specific ROM directories (atari2600, arcade, etc.)
- Moved ROM files to appropriate directories using mv command
- Command: mv ~/RetroPie/roms/*.zip ~/RetroPie/roms/atari2600/



[Fig 13: SSH Terminal - File Organization]

4.3 System Configuration

Performance Optimization

- GPU Memory Split: Allocated 256MB to GPU (gpu_mem=256 in /boot/config.txt)
- CPU Governor: Set to 'performance' mode for consistent clock speeds
- Disabled unnecessary system services to reduce background processes

5. Implementation Challenges

5.1 Storage Media Limitations (microSD Card Issues)

One of the major challenges faced during the implementation was the use of a low-quality microSD card. The Raspberry Pi Imager repeatedly failed to flash the RetroPie operating system image onto the SD card, even after multiple attempts. This issue highlighted the sensitivity of the flashing process to storage media quality, particularly when writing large OS images.

To overcome this issue, alternative flashing software (BalenaEtcher) was used. Although BalenaEtcher also failed in initial attempts, repeated retries eventually resulted in a

successful image write. However, the impact of the low-quality SD card persisted beyond installation.

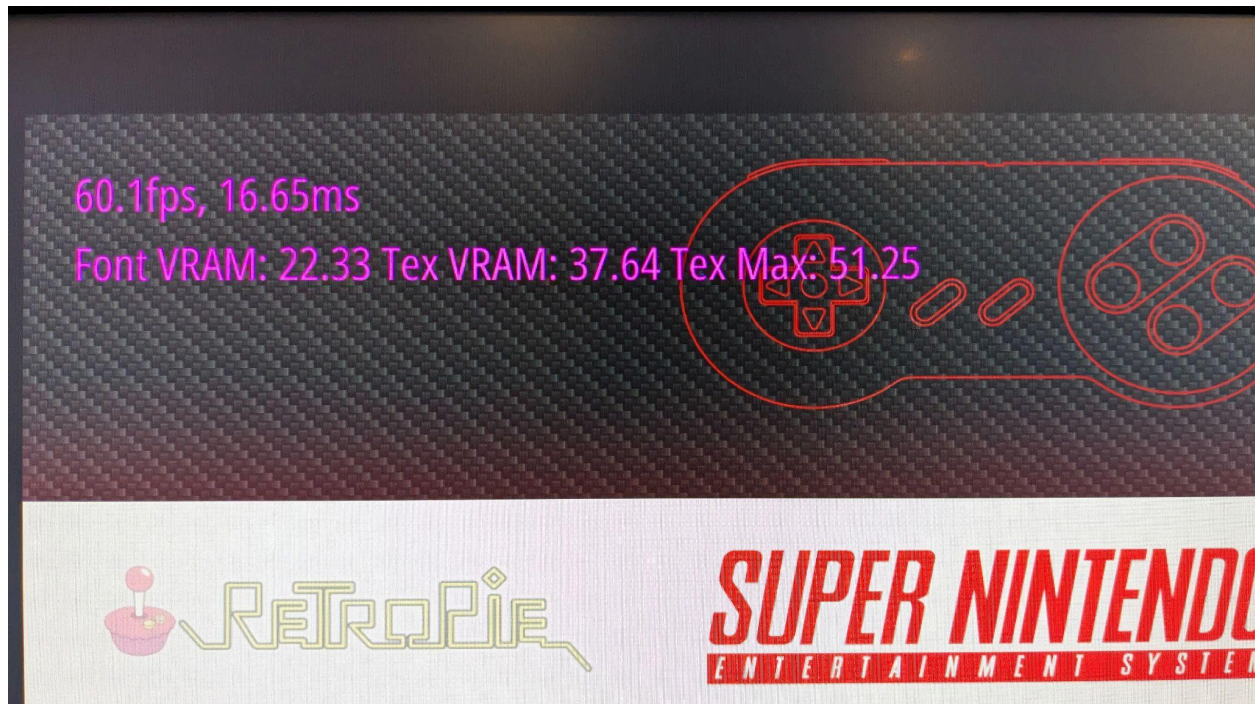
During system usage and gameplay, frequent screen flickering, display signal loss, and intermittent blackouts were observed. These issues occurred due to slow read/write speeds and unstable data access from the microSD card, directly affecting I/O performance and system stability. This demonstrated how storage hardware plays a critical role in operating system reliability, boot consistency, and real-time application performance on embedded systems like Raspberry Pi.

Challenge	Solution	OS Concept
Under-voltage warnings during boot	Replaced with 5V/3A certified power supply	Hardware abstraction, power management
WiFi configuration failure	Used raspi-config to manually configure network settings	Network drivers, device configuration
Slow ROM loading times	Optimized file system performance, increased GPU memory allocation	File system I/O, memory management
Controller input lag	Configured RetroArch input latency settings, reduced audio buffer	I/O management, interrupt handling

6. Testing and Performance Metrics

6.1 System Performance

Metric	Measurement
Boot Time (Power-on to EmulationStation)	~18 seconds
Average CPU Usage (Idle)	15-20%
Average CPU Usage (Gaming)	45-65%
RAM Usage (System + EmulationStation)	~550MB / 3.7GB available
Operating Temperature (Under Load)	40-48°C (CPU), 40-48°C (GPU)
Frame Rate (Atari 2600 Emulation)	Stable 60 FPS



[Fig 14: System Monitoring (<http://Temperature>)]

6.2 Functional Testing

- Controller input responsiveness: All buttons mapped and functioning correctly
- ROM loading: Games load within 2-3 seconds
- Save/Load states: Functionality verified across multiple games
- Network file transfer: SSH and Samba services operational



[Fig 15: Game Running (EmulationStation)]

7. Learning Outcomes

7.1 Technical Knowledge Gained

- Practical understanding of Linux boot process and systemd initialization
- Hands-on experience with process management and resource allocation
- Configuration of device drivers and hardware abstraction layers
- File system management and organization in Linux environment
- System service administration (SSH, Samba, network configuration)
- Performance optimization through OS-level configuration

7.2 Operating System Concepts Applied

- Layered OS architecture (hardware, kernel, services, applications)
- Process scheduling and multi-threading in real-time applications
- Memory management techniques (allocation, virtual memory, caching)
- I/O subsystem design for low-latency input processing
- Hardware-software interface through device drivers

8. Conclusion

The RetroPie gaming console project successfully demonstrates core operating system concepts through practical implementation. The project provided hands-on experience with Linux system administration, process management, memory optimization, file system organization, and hardware-software integration. Key achievements include successful OS installation and configuration, optimization of system performance for real-time gaming, implementation of network services, and comprehensive understanding of layered OS architecture. The project reinforced theoretical OS knowledge with practical skills in embedded Linux environments and system-level programming.

8.1 Future Enhancements

- Implementation of automatic ROM metadata scraping system
- Configuration of network-based multiplayer functionality
- Integration of additional emulator cores for expanded system support
- Implementation of automated backup system for save states and configurations

9. References

- RetroPie Official Documentation: <https://retropie.org.uk/docs/>
- Raspberry Pi Documentation: <https://www.raspberrypi.org/documentation/>
- Linux Kernel Documentation: <https://www.kernel.org/doc/>
- Operating Systems: Three Easy Pieces by Remzi H. Arpaci-Dusseau

10. Appendix

Command	Purpose
<code>sudo raspi-config</code>	System configuration utility
<code>ping 192.168.1.14</code>	Network connectivity test
<code>ssh pi@192.168.1.14</code>	Remote terminal access
<code>scp -r files/ pi@IP:~/RetroPie/roms/</code>	Secure file transfer
<code>mkdir -p ~/RetroPie/roms/system</code>	Create ROM directories
<code>sudo systemctl status nmbd</code>	Check Samba service status
<code>htop</code>	Monitor system resources